

Introduction to Java & Its History

1. The Evolution of Programming (Why Java?)

To understand why Java is so popular, we must look at the timeline of innovation. Every language was created to solve a specific problem of its time.

1972 → C Language (The Foundation)

- **The Goal:** Built to write Operating Systems (like UNIX).
- **The Limitation:** C is **Platform Dependent**. If you write a program on Windows, it cannot run on Linux without rewriting or recompiling.
- **Summary:** Great for hardware, but not flexible enough for networks.

1990 → The Rise of the Internet (WWW)

- **The Context:** The World Wide Web connected different types of computers globally.
- **The Problem:** We needed a language that could travel across the internet and run on *any* computer, regardless of whether it was a Mac, PC, or Mainframe.

1995 → Java (The Revolution)

- **The Solution:** Released by Sun Microsystems.
- **The Breakthrough:** Java introduced **WORA (Write Once, Run Anywhere)**.
- **How it Works:** Unlike C, Java compiles into **Bytecode**. This bytecode runs inside the **JVM (Java Virtual Machine)**, which makes the code neutral to the underlying hardware.

2. Understanding Java Editions (The "J2" Era)

In the late 90s (specifically starting with version 1.2), Java was rebranded as **Java 2**. This is why you will often see terms starting with "J2" in older documentation and legacy systems.

What does "J2" stand for?

It stands for **Java 2**. It refers to the massive platform update around the year 2000 that split Java into three distinct editions to serve different needs.

The Three Editions:

1. J2SE (Java 2 Standard Edition)

- **Modern Name:** Java SE (Standard Edition).
- **What it is:** This is "Core Java." It provides the essential functionality of the language.
- **Key Components:** `java.lang`, `java.util`, Collections, Streams, Multithreading.
- **Usage:** Used for developing desktop applications, command-line tools, and learning the fundamentals.

2. J2EE (Java 2 Enterprise Edition)

- **Modern Name:** Jakarta EE (formerly Java EE).
- **What it is:** This is "Advanced Java." It is built *on top* of J2SE to create large-scale server applications.
- **Key Components:** Servlets, JSP (JavaServer Pages), EJB (Enterprise JavaBeans).
- **Usage:** Used for banking systems, e-commerce backends, and web portals.
- *Note:* In modern development, frameworks like **Spring Boot** are used to implement J2EE concepts more easily.

3. J2ME (Java 2 Micro Edition)

- **Modern Name:** Java ME.
- **What it is:** A lightweight version of Java optimized for devices with limited memory and processing power.
- **Usage:** Originally used for old mobile phones (Nokia/Symbian apps), set-top boxes, and embedded chips.

3. Industry Terminology: Core vs. Advanced Java

While Oracle (the creators of Java) does not officially split the language, the training and job market divides it into two categories:

Category	Topics Covered	Real-World Focus
Core Java	Variables, OOPs, Collections, Exception Handling	Building the Logic of the application.
Adv. Java	JDBC, Servlets, JSP, Web Services	Building the Server & Database connectivity.

Topic: Introduction to Database Applications & JDBC

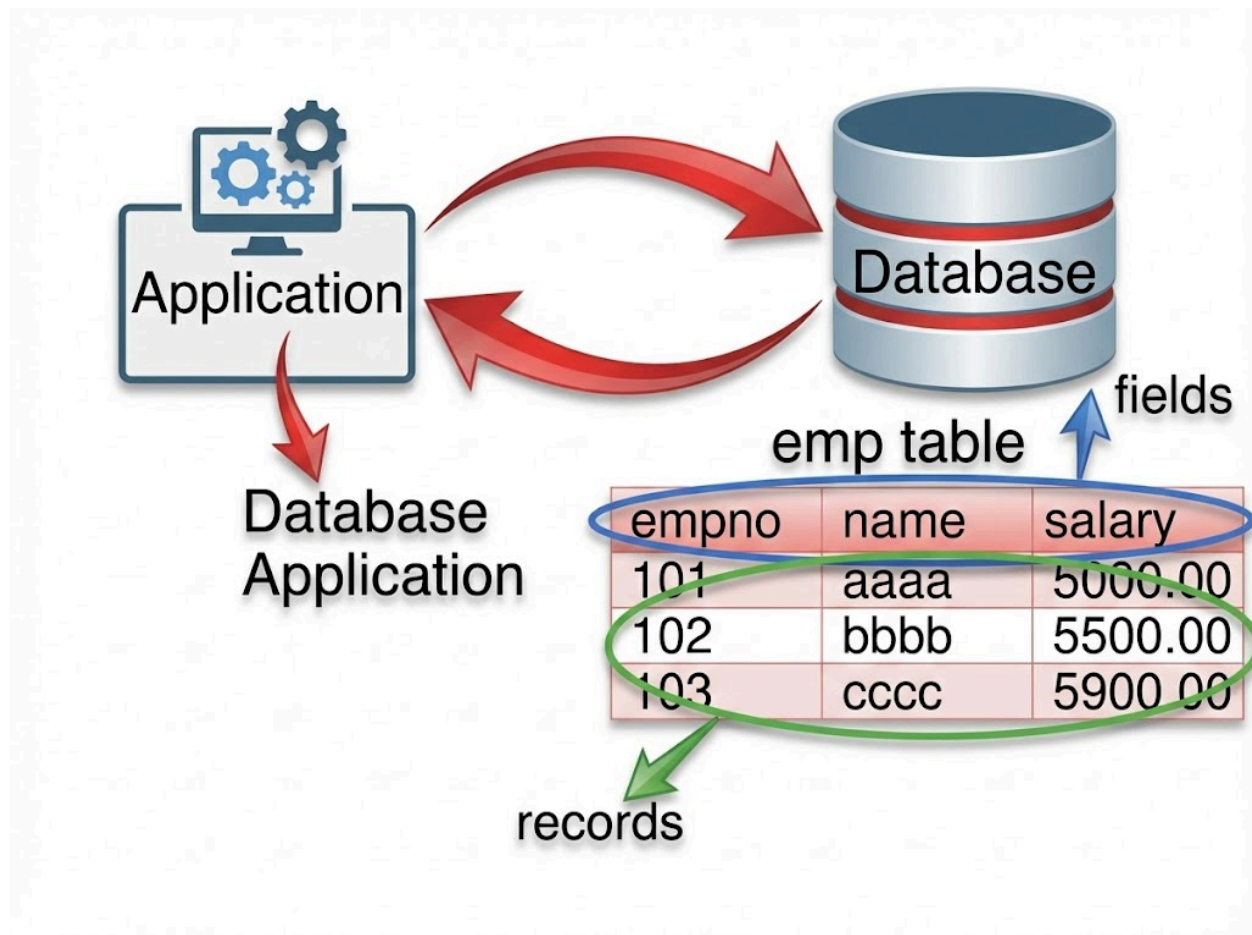
1. Basic Terminology

- **Application:**

- Definition: "An application is a program in which we interact with on the desktop."
- Database:
 - Definition: "A database is a software and it is an organized collection of data."
 - *Key Concept*: It is the software (DBMS) that manages the storage.
- Database Application:
 - Definition: "An application that communicates with a database is known as database application."
 - *Example*: Any program that saves or retrieves permanent data.

2. JDBC (Java DataBase Connectivity)

- Full Form: JDBC stands for Java DataBase Connectivity.
- Core Definition: "JDBC is a specification for developing database applications with Java programming language."



Database software contains two part:

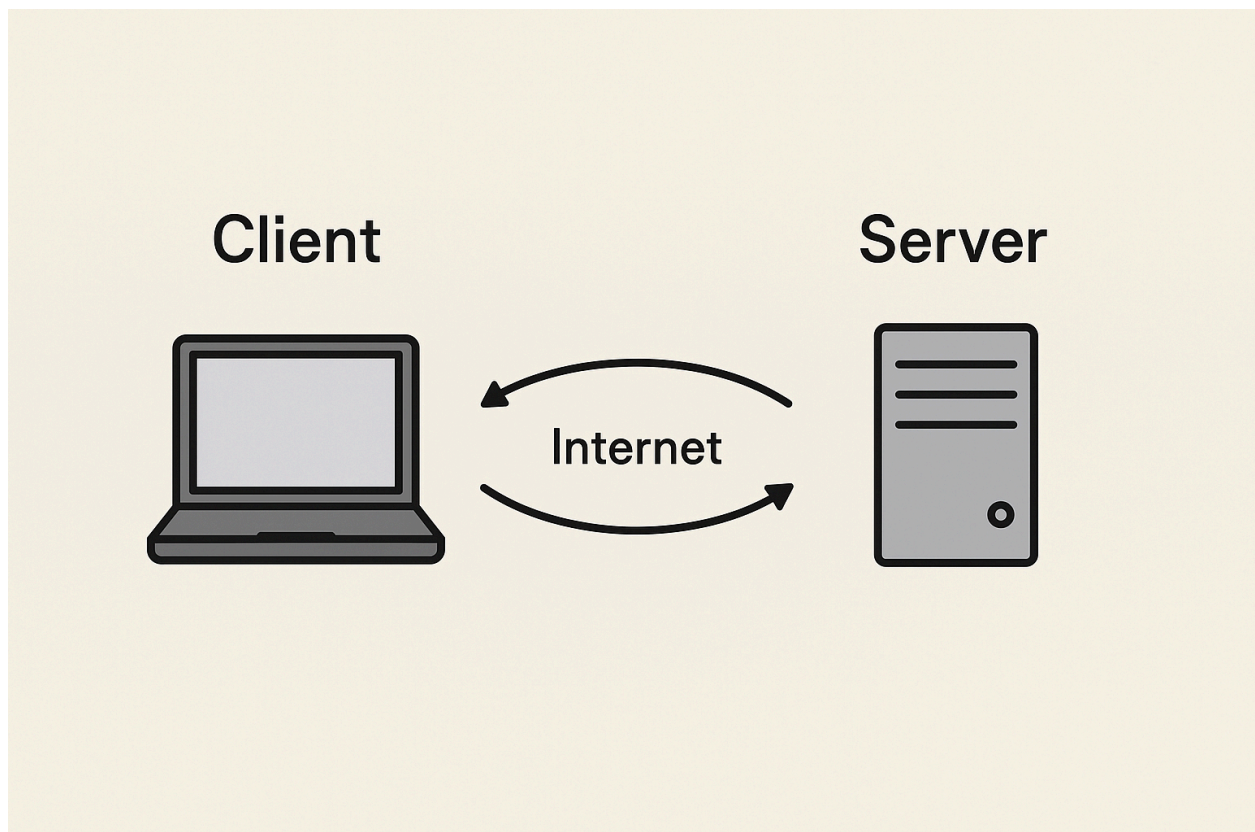
1. Database application (Example: SQL Prompt).
2. Database (Example: MySQL).

Notes: Database application is front end, whereas database is a back end. Applications are communicating with databases by using SQL.

SQL:

SQL stands for structure query language and it is used by applications to communicate databases.

Client-Server Architecture & The Need for Connectivity.



1. The Client-Server Model (Official Definitions)

Before understanding databases, we must understand how two softwares talk to each other.

- **Client:**
 - "A client is a software that sends the request to a server to get the response."
- **Server:**
 - "A server is a software that receives a request from the client, processes the request, constructs the response & sends the response back to the client."

1. The Client-Server Analogy (Database vs. App)

Reference: Database Software Parts

To understand how these two parts work, we treat them as a Client-Server model:

- **The Client (Frontend/Requestor):**
 - This is the **Database Application** (e.g., SQL Prompt, Java App).
 - **Role:** It sends a *request* (a query) to get data.
- **The Server (Backend/Provider):**
 - This is the Database software (e.g., Oracle, MySQL).
 - **Role:** It receives the request, processes it, and sends back the *response* (data).

2. The "Agra vs. Delhi" Scenario (Why we need Connectivity)

Reference: Database Application Definition

Case A: The Local System (Same Room)

- If the CMD and the Database are on the same laptop, they can talk easily.

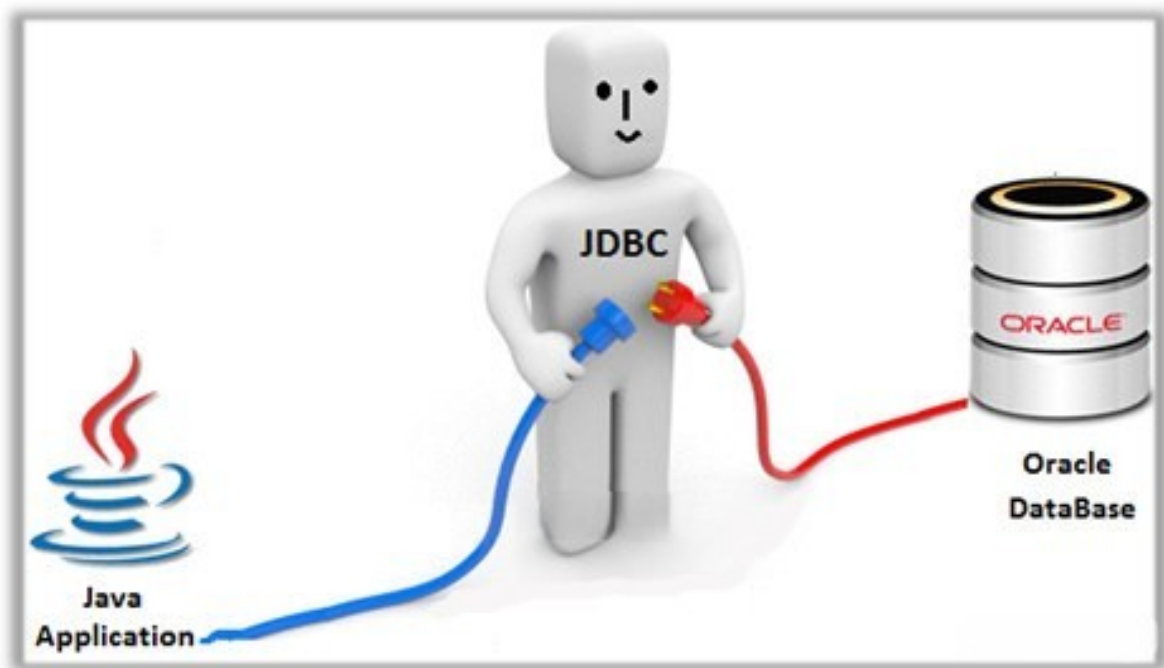
Case B: The Real World (Agra vs. Delhi)

- Imagine your CMD is running on a laptop in Agra.
- But your Database (Oracle) is installed on a server in Delhi.

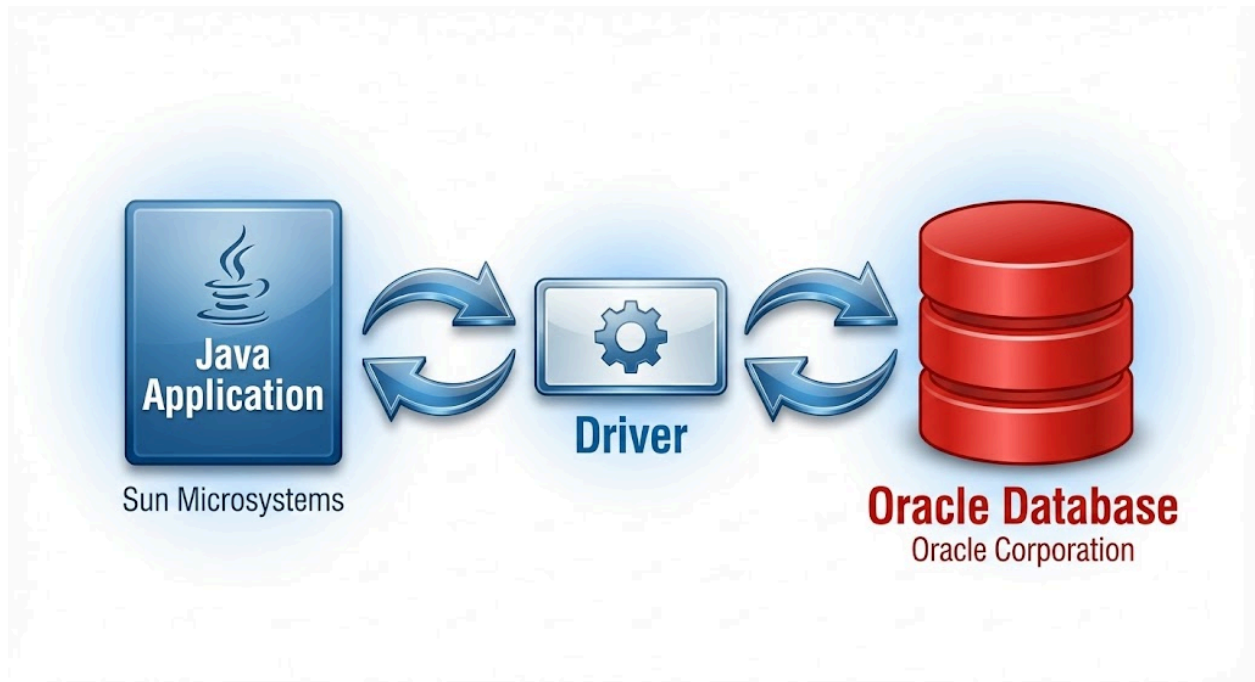
- The Problem: Can the App in Agra directly touch the hard drive in Delhi? No.
- The Solution: This is why we need Communication. We need a "phone line" or a "bridge" between Agra and Delhi.
 - That bridge is the Network.
 - The language spoken over that bridge is JDBC.

Driver:

A driver is software and it is used to connect applications and databases.



Drivers are developed by first party vendors, second party vendors & third party vendors.



Understanding "Vendors" (Who owns what?)

In the software industry, we classify companies based on their relationship to the technology.

A. First Party Vendor (The Creator of the Language)

- Definition: The company that designed the programming language and the *Specification* (rules) for connection.
- In this example: Sun Microsystems.

B. Second Party Vendor (The Creator of the Database)

- Definition: The company that creates the database software we are trying to connect to.
- In this example: Oracle Corporation.

C. Third Party Vendor (The External Facilitator)

- Definition: Any company *other than* the First Party (Sun) and Second Party (Oracle) that creates tools or drivers to help them connect.

Specification:

The Core Definition

- What is it?
 - "A specification is a set of rules & guidelines that are used to develop applications & environments."
- In Simple Terms: Think of a Specification as a Rulebook or a Blueprint. It doesn't tell you *how* to code it, but it tells you *what* the code must do to be correct.

2. Visualizing the Concept

The diagram illustrates how the Specification sits at the top of the hierarchy:

- The Authority: The Specification is the central document containing the rules.
- The Followers:
 - Java Compiler: It must be built according to these rules so it knows how to read your code.
 - Java Program: The code you write must follow these rules so the compiler can understand it.



NOTE: JDBC Specification used by vendor to develop driver

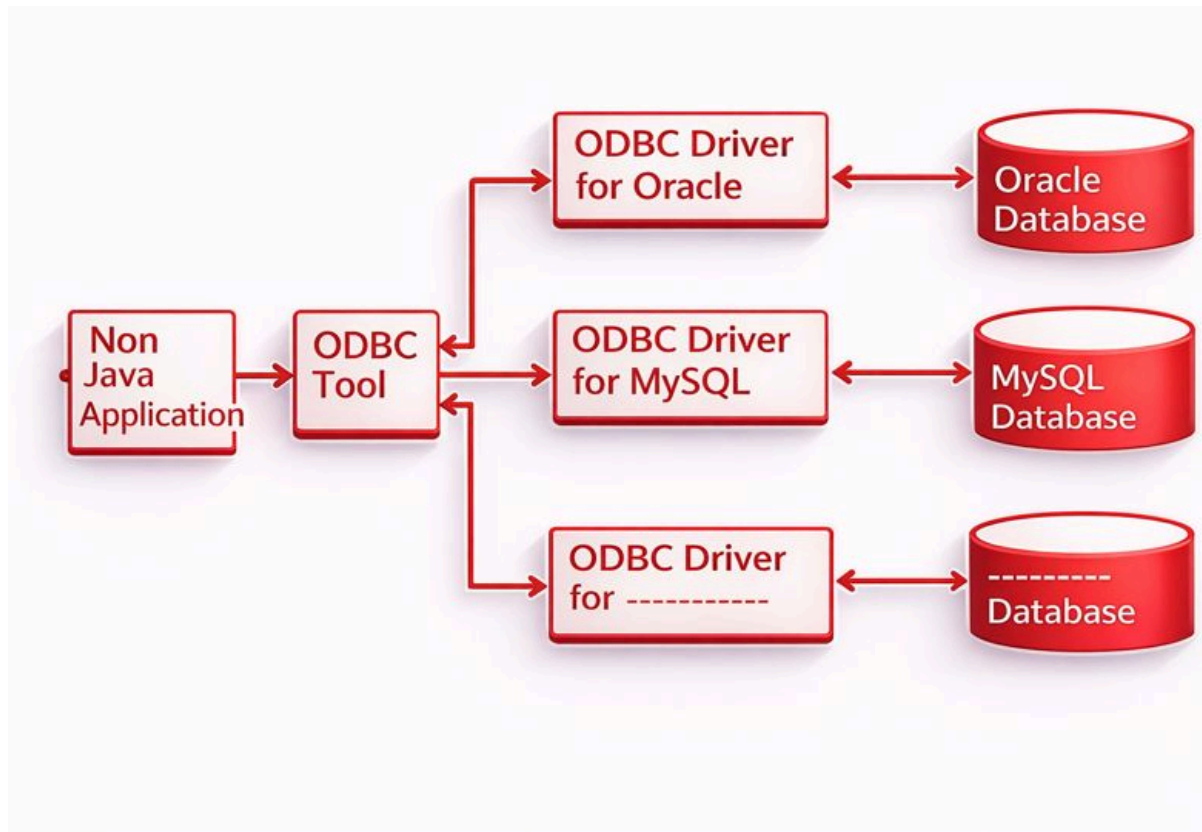
JDBC specification used by Java Programmers to develop database applications.

There are two types of driver:

1. ODBC(open database connectivity) drivers
2. JDBC(Java database connectivity) drivers

ODBC is used to connect non java applications & databases.

ODBC driver exists before java



This diagram shows that **ODBC is language-independent** because the application layer is abstracted from the database using a common ODBC interface. The application, regardless of the programming language, communicates only with the ODBC tool, while database-specific communication is handled by ODBC drivers. Hence, neither the application language nor the database type affects each other.

All odbc drivers are part of the odbc tool.

Why JDBC

- Java cannot directly use ODBC
- ODBC does not understand Java instructions
- JDBC was introduced to connect Java applications with databases

JDBC Drivers

There are 4 types of JDBC drivers:

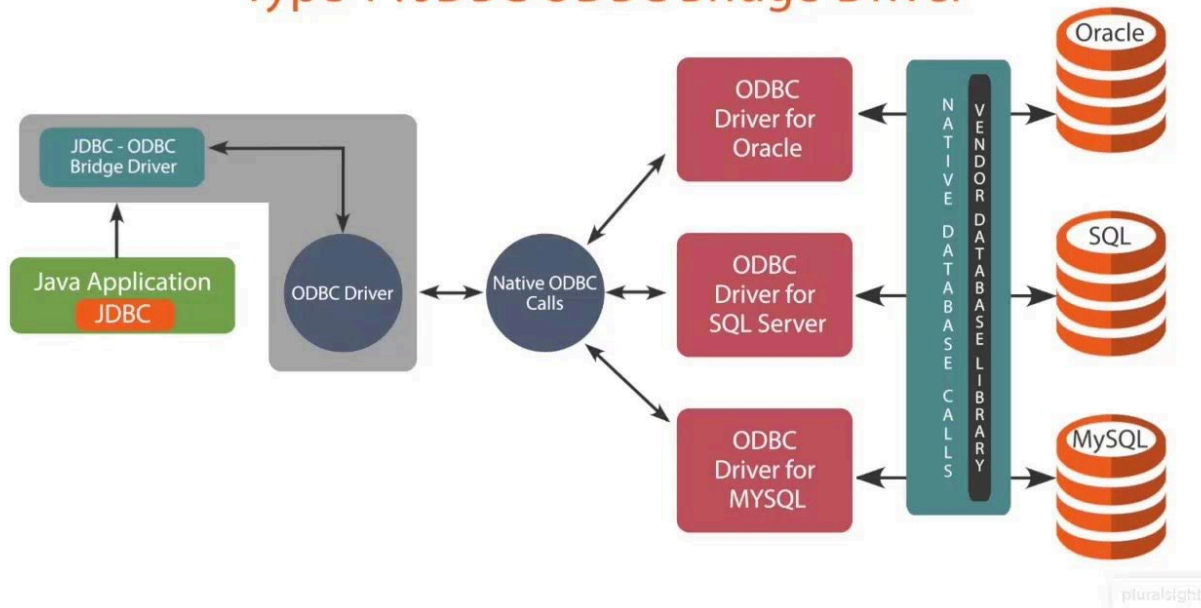
- 1 Type 1 – JDBC-ODBC Bridge (Deprecated)
- 2 Type 2 – Native API Driver
- 3 Type 3 – Network Protocol Driver
- 4 Type 4 – Pure Java (Thin Driver, Most Used)

Type 1 – JDBC-ODBC Bridge (Deprecated)

Driver Class Name: `sun.jdbc.odbc.jdbcodbcDriver`.

Java to database connection failed so this came in picture, removed from java 1.8

Type 1 : JDBC ODBC Bridge Driver



1. Java Application (JDBC)

- Your Java program
- Does not know anything about ODBC or database internals

◆ 2. JDBC-ODBC Bridge Driver (Type 1)

- A special JDBC driver
- Converts JDBC calls → ODBC calls
- Acts as a bridge between Java and ODBC

◆ 3. ODBC Driver

- Database-specific ODBC driver

- Examples:
 - ODBC Driver for Oracle
 - ODBC Driver for SQL Server
 - ODBC Driver for MySQL
- Receives ODBC calls from the bridge

◆ 4. Native ODBC Calls

- Low-level, native (C/C++) calls
- Used internally by ODBC drivers
- This is why Type-1 is platform-dependent

◆ 5. Vendor Database Library

- Database vendor's native client libraries
- Example:
 - Oracle client
 - SQL Server native client
- Translates calls into database protocol

◆ 6. Database (Oracle / SQL Server / MySQL)

- Actual database where data is stored
- Executes SQL queries
- Sends result back through the same path

multiple layers → slow performance

Type 2 – Native API Driver:

It is also called partly Java, partly Native driver or partial Java driver.

Type 2 drivers developed in native language & java language.

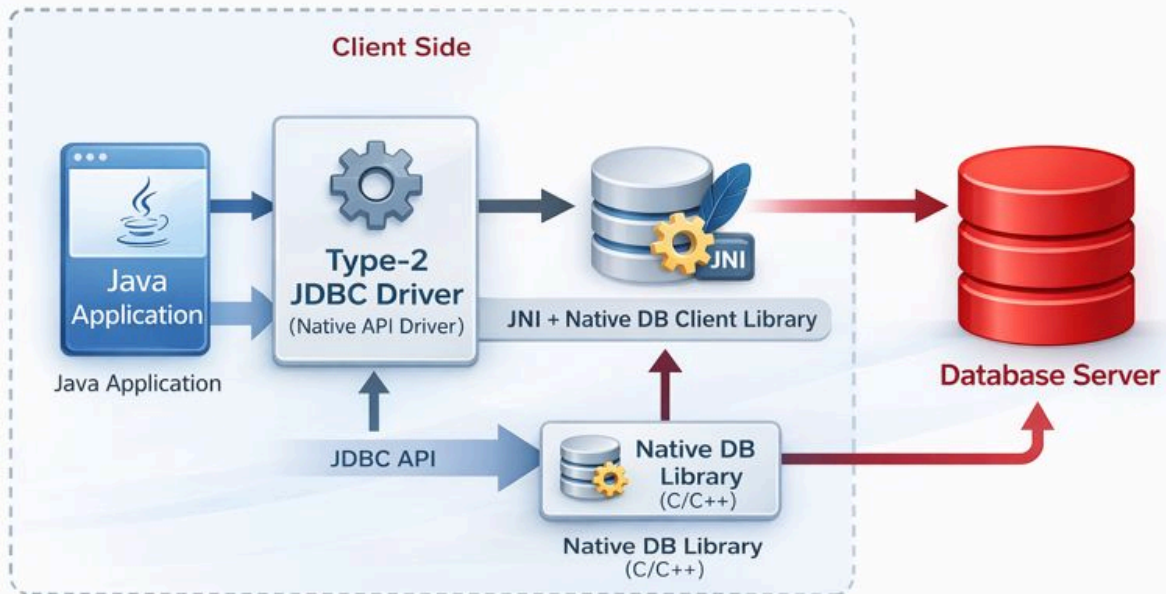
Class name: `com.mysql.cj.jdbc.Driver`, `oracle.jdbc.driver.OracleDriver`

Why do JDBC drivers have different classes?

Note:

JDBC drivers have different classes because each database uses a different communication protocol and vendor-specific features, requiring separate implementations of the `java.sql.Driver` interface.

Type-2 (Native API) JDBC Driver



First understand the problem JDBC tries to solve

Java programs cannot talk directly to a database.

Why?

- Java speaks Java language
- Database speaks its own language / protocol

So we need a translator.

That translator is called a JDBC Driver.

What is Type-2 JDBC Driver?

Type-2 JDBC Driver uses **native database code (written in C / C++)** to talk to the database.

In simple words:

- Java does not talk to databases directly.
- Java talks to native DB software, and that software talks to databases.

That's why it is called Native-API Driver.

Example (Oracle Type-2 Driver)

Driver used:

OCI Driver (Oracle Call Interface)

URL:

`jdbc:oracle:oci:@ORCL`

⚠ If Oracle Client is NOT installed, application will fail.

Note:

- Java Application sends SQL queries using JDBC API.
- Type-2 JDBC Driver receives these calls and prepares them for native execution.
- JNI acts as a bridge to convert Java calls into C/C++ function calls.
- Native DB Client Library (C/C++) communicates using database-specific protocol.

- Database Server executes the query and returns the result back through the same path.

Type-2 JDBC Driver converts JDBC calls into database-specific native calls using native libraries. It offers better performance than Type-1 but is platform dependent and requires database client installation.

Java Application



Type-2 JDBC Driver (Java)



JNI



Native DB Client Library (C/C++)



Database Server

Type-3 JDBC Driver (Network Protocol Driver)

Type-3 JDBC Driver sends JDBC calls to a middleware server, and that server talks to the database.

Java Application



JDBC API



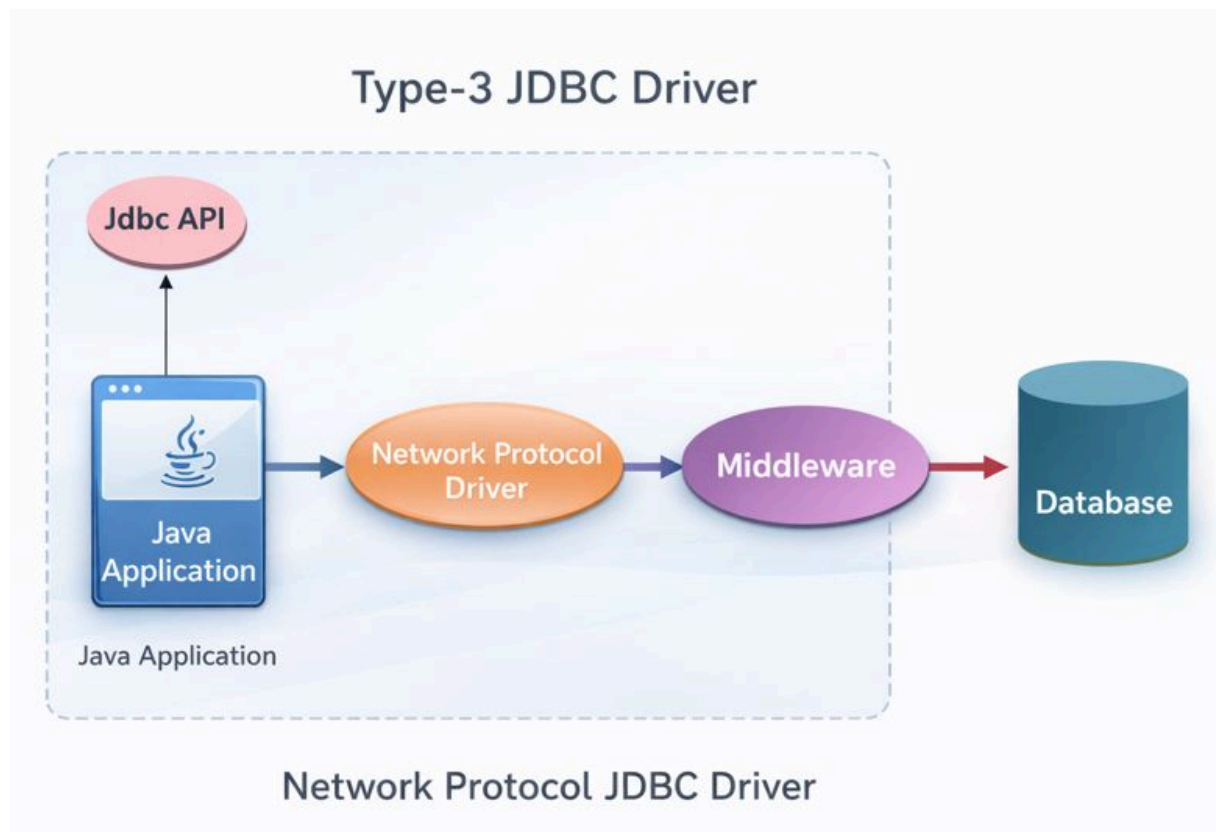
Type-3 JDBC Driver



Middleware Server (Network Protocol)



Database Server



Type-4 JDBC Driver (Thin / Pure Java Driver)

Type-4 JDBC Driver allows Java to talk DIRECTLY to the database using pure Java code.

Java Application



JDBC API



Type-4 JDBC Driver (Pure Java)



Database Server

Simple real-life example 🧠

Think like this:

- Java = You
- Database = Shopkeeper

You talk directly to the shopkeeper.
No agent, no translator, no middleman.-

Type-4 JDBC Driver

(Pure Java Driver / Thin Driver)

